

DEOK - HUGAM

누군가의 기록이 데이터가 되는 곳

팀명 read-compass
주제 덕후감 프로젝트
깃허브 [github.com/sb12-
Read-Compass-team-RC](https://github.com/sb12-Read-Compass-team-RC)



#도서 등록 read_compass



사진을 올리기만 했는데 ISBN로 책 정보가 매칭되는
경험이 신선했어요.
책 정보 등록이 이렇게 쉬울일인가...

♡ 68 💬 22

26.07.10



#리뷰/댓글 작성 read_compass



리뷰를 작성하고 댓글로 바로 의견을 나눌 수 있어서
좋았어요. 혼자 읽은 책인데도 같이 이야기하는 느낌
이 들었습니다.

♡ 87 💬 123

26.07.23



#리뷰/도서/사용자 랭킹 read_co... ★★★★★

리뷰와 도서, 사용자 랭킹을 따라가다 보니 좋은 글과
좋은 책을 자연스럽게 만나게 됐어요.
이제는 책을 고르는 시간이 설렘으로 다가오네요...

♡ 11 💬 57

26.08.15

CONTENTS

01

개요

프로젝트 소개

02

기획

프로젝트 선정 이유
기술 스택
프로젝트 진행 일정

03

분석

API 명세 요약
기능 리스트
역할 분담
협업 방식

04

설계

ERD
핵심 기능 설계
시스템 구조도

05

구현

주요 기능
시연
트러블 슈팅

1. 프로젝트 소개

01. OVERVIEW



‘덕후감’이란?

책을 좋아하는 ‘책덕후’들이 모여 감상을 나누는 서비스를 의미합니다.
Naver Open API의 ISBN 매칭과 OCR Space API의 도서 이미지 OCR을
통해 도서의 정보를 쉽게 불러오고 평가와 감상을 나눌 수 있습니다.

Team

장현우 전민지 정우진 이지예 김성규

Period

2026.06.22 ~ 2026.07.13

Fuction

도서	도서 CRUD · OCR
리뷰	리뷰 CRUD · 좋아요
댓글	댓글 CRUD
알림	리뷰 좋아요 · 댓글 알림
대시보드	도서 · 리뷰 랭킹 조회
배치 시스템	도서 · 리뷰 · 파워 유저랭킹 계산

Open API



NAVER OpenAPI
OCRSpace

Tool, Framework & Language

Backend



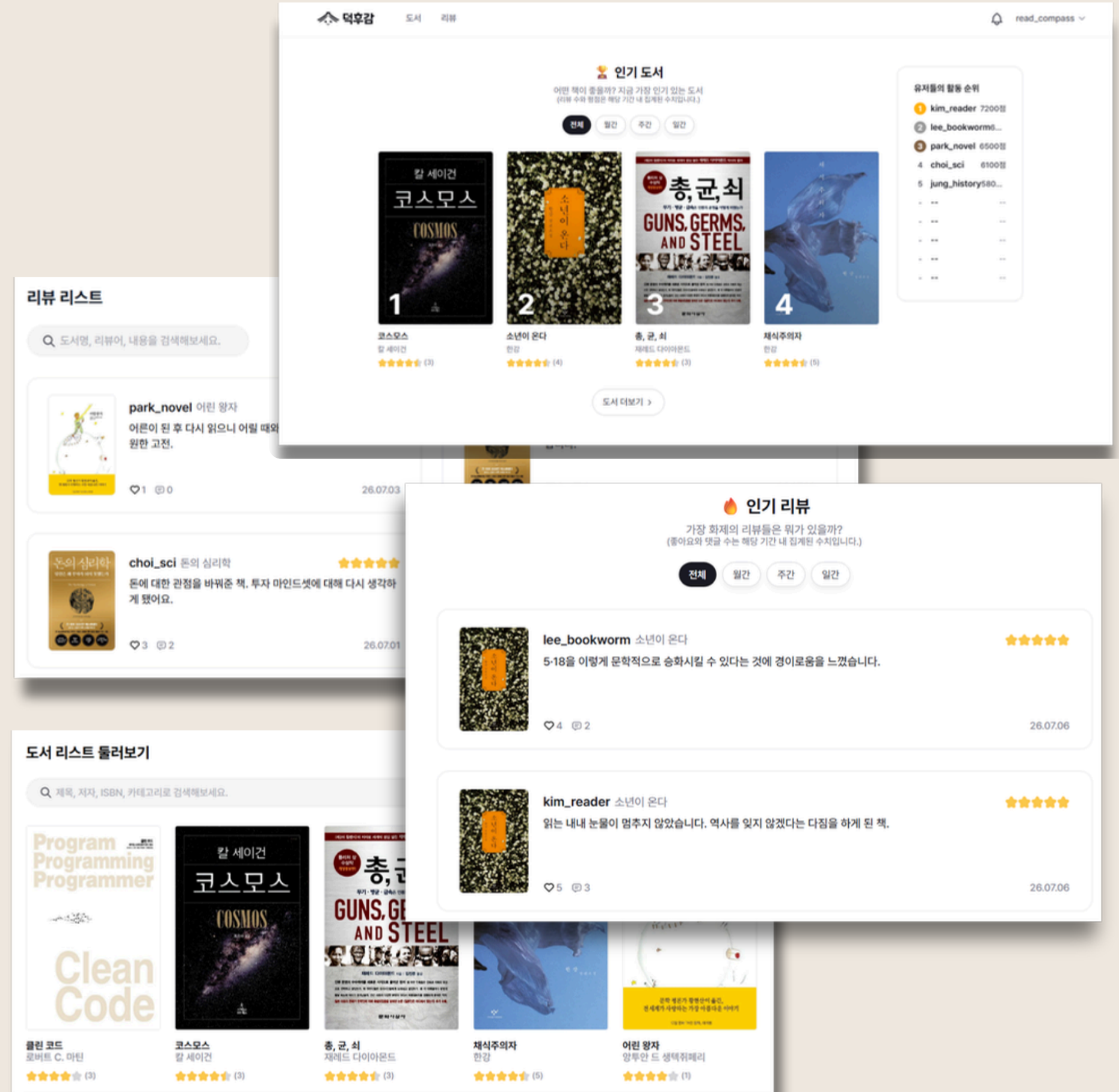
Frontend



Authorization / Authentication



Realease



1. 프로젝트 선정 이유

2. 기술 스택

3. 프로젝트 진행 일정

02. CONCEPT

프로젝트 선정 이유



01 문제발견

같은 책을 읽은 사람들과 물리적,
시간적 제약으로 감상을 나누기
어려운 문제발견



02 아이디어

시간과 장소의 제약없이 독서경험을
공유하고 소통할 수 있는 온라인 커
뮤니티를 개발



03 목표

책 등록과정을 간소화해 누구나 부담
없이 리뷰를 작성하고 소통하는 것



DEOK-HUGAM

지금 쓰는 책



이 책을 읽은 사람들

 **minji**
2시간 전

“이기적 유전자”를 읽고

혼자 읽은 책인데도 같이 이야기하는
느낌이 들었어요.



최종목표

독서를 혼자가 아닌 함께하는 경험으로 만들고,
누구나 쉽게 책을 등록하고 리뷰를 나눌 수 있는 서비스를 제공합니다

#도서기록

#리뷰공유

#커뮤니티

#소통

기술 스택

백엔드



JAVA



Spring Boot



JPA



QueryDSL



Gradle

인증/인가



Spring Security



JWT



OAuth2

인프라 /배포



AWS



Docker



S3



GitHub Actions

외부 API

OCRSpace **NAVER** OpenAPI

Google Cloud Platform

협업/개발 도구



GitHub



Git



IntelliJ



Notion



Jira



Discord

데이터베이스



PostgreSQL



RDS

프로젝트 진행 일정

	WEEK 1 6.22 - 6.28							WEEK 2 6.29 - 7.5							WEEK 3 7.6 - 7.12						
TASKS	22	23	24	25	26	27	28	29	30	1	2	3	4	5	6	7	8	9	10	11	12
기 획	6.22 - 6.24																				
분 석		6.23 - 6.25																			
설 계				6.25																	
서비스 기능 구현				6.25 - 7.3																	
배 치 기능 구현									6.30 - 7.2												
테스트 / 버그 수정												7.3 - 7.6									
배 포															7.6 - 7.8						
포트폴리오 작성																		7.8 - 7.12			

03. ANALYZING

- 1.API 명세 요약
- 2.주요 기능 및 요구사항
- 3.팀 구성 및 역할 분담
- 4.협업 방식

API 명세 요약

사용자 관리 회원/인증

회원가입	POST	/api/users
로그인	POST	/api/users/login
사용자 정보 조회	GET	/api/users/{userId}
사용자 정보 수정	PATCH	/api/users/{userId}
사용자 논리 삭제	DELETE	/api/users/{userId}
사용자 물리 삭제	DELETE	/api/users/{userId}/hard
도서 목록 조회	GET	/api/books

도서 관리 도서/ISBN

도서 목록 조회	GET	/api/books
도서 등록	POST	/api/books
OCR 기반 ISBN 인식	POST	/api/books/isbn/ocr
도서 상세 정보 조회	GET	/api/books/{bookId}
도서 정보 수정	PATCH	/api/books/{bookId}
도서 논리 삭제	DELETE	/api/books/{bookId}
ISBN으로 도서 정보 조회	GET	/api/books/info
도서 물리 삭제	DELETE	/api/books/{bookId}/hard

리뷰 관리 리뷰/좋아요

리뷰 목록 조회	GET	/api/reviews
리뷰 등록	POST	/api/reviews
리뷰 좋아요	POST	/api/reviews/{reviewId}/like
리뷰 상세 정보 조회	GET	/api/reviews/{reviewId}
리뷰 수정	PATCH	/api/reviews/{reviewId}
리뷰 논리 삭제	DELETE	/api/reviews/{reviewId}
리뷰 물리 삭제	DELETE	/api/reviews/{reviewId}/hard
리뷰 댓글 목록 조회	GET	/api/comments

공통 API 규칙

구분	내용
인증 방식	헤더(엑세스 토큰) 쿠키(리프레시 토큰)
인증 사용자 식별	AuthenticationPrincipal 어노테이션
페이지네이션	cursor after limit
정렬 방향	DESC ASC
에러 응답	상태 코드 + 메세지 반환

댓글 관리 댓글

리뷰 댓글 목록 조회	GET	/api/comments
댓글 등록	POST	/api/comments
댓글 상세 정보 조회	GET	/api/comments/{commentId}
댓글 수정	PATCH	/api/comments/{commentId}
댓글 논리 삭제	DELETE	/api/comments/{commentId}
댓글 물리 삭제	DELETE	/api/comments/{commentId}/hard

알림 관리 알림

알림 목록 조회	GET	/api/notifications
알림 읽음 상태 업데이트	PATCH	/api/notifications/{notificationId}
모든 알림 읽음 처리	PATCH	/api/notifications/read-all

대시보드 관리 랭킹

인기 도서 목록 조회	GET	/api/books/popular
인기 리뷰 목록 조회	GET	/api/reviews/popular
파워 유저 목록 조회	GET	/api/users/power

주요 기능 및 요구사항



Spring Batch

랭킹 점수 계산

파워 유저 점수 계산

인기 도서점수 계산

인기 리뷰 점수 계산

핵심 요구사항

- 도서 등록·검색 및 리뷰 작성 기능 제공
- ISBN·OCR 기반 도서 정보 자동 입력
- 댓글·좋아요·알림을 통한 사용자 상호작용
- 인기 도서·리뷰·파워 유저 랭킹 제공
- 안전한 인증 및 권한 관리(JWT/OAuth2)

팀 구성 및 역할 분담

Team : Read Compass

**장현우**

팀장

대시보드 및 인증 관리

- 대시보드 랭킹 기능 개발
- 인증 및 인가 기능 구현
- 배치 기반 집계 기능 구현
- 서비스 배포 및 운영 환경 관리

github.com/gusdn6763**전민지**

팀원

도서 관리

- 도서 CRUD 기능 개발
- ISBN 기반 도서 정보 조회 구현
- OCR 및 S3 썸네일 기능 연동
- 도서 카테고리 기능 추가

github.com/jeon-minji**이지예**

팀원

댓글 관리

- 댓글 CRUD 기능 개발
- 댓글 조회 및 페이지네이션 구현
- 알림 연동 데이터 처리

github.com/yecoder514**김성규**

팀원

사용자 관리

- 사용자 CRUD 기능 개발
- 회원가입 및 로그인 기능 구현
- 사용자 논리 삭제 기능 구현
- DB 스키마 설계 및 최적화

github.com/ksg-97**정우진**

팀원

리뷰 및 좋아요 관리

- 리뷰 CRUD 기능 개발
- 리뷰 수정 및 삭제 권한 처리
- 리뷰 좋아요 기능 구현
- API 명세서 작성 관리

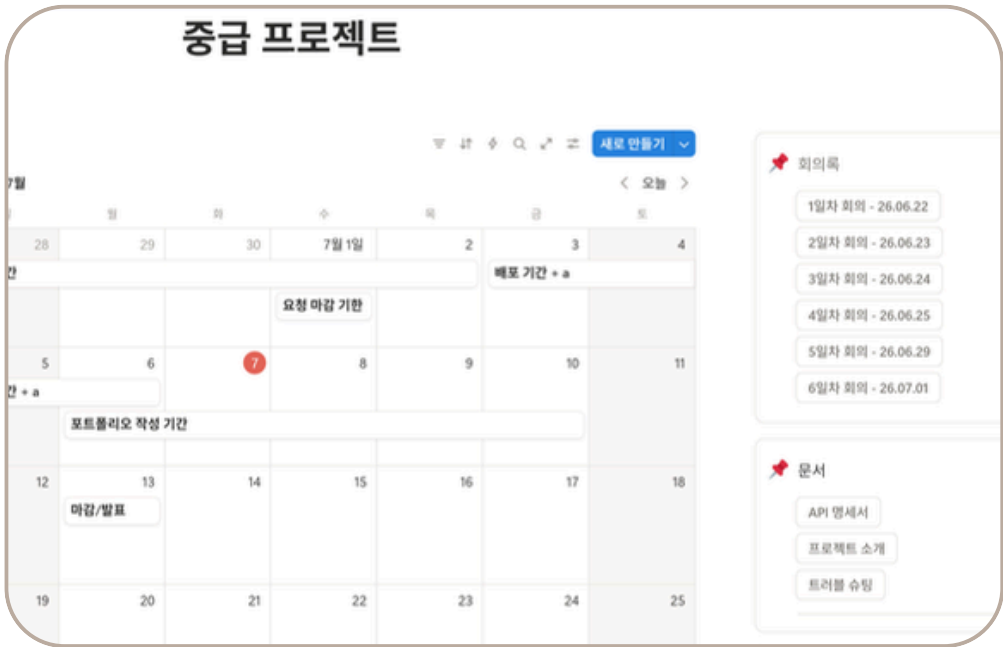
github.com/zinzin68

협업 방식



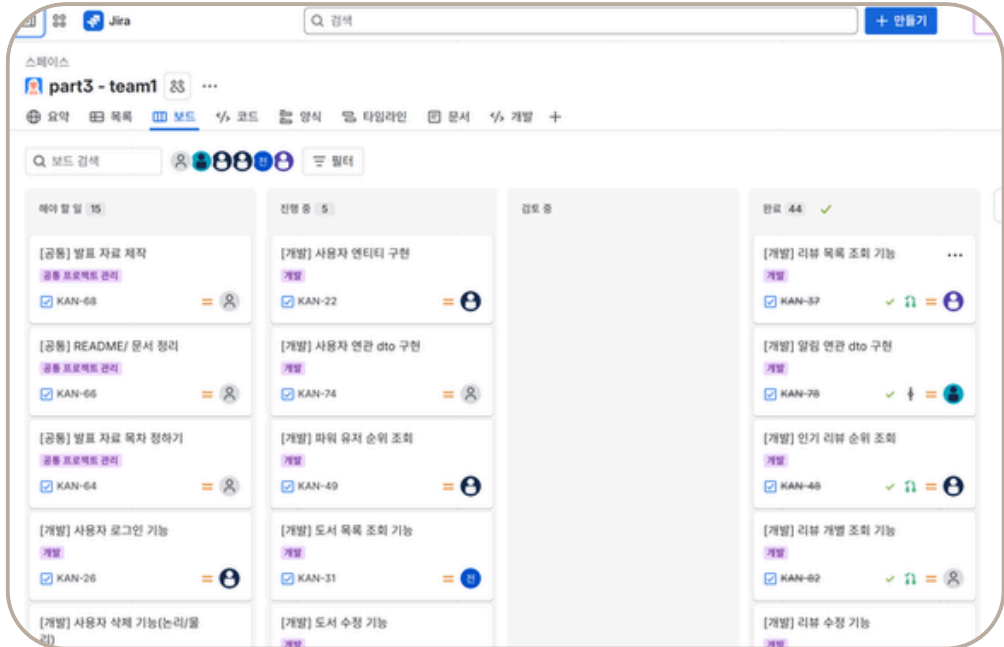
프로젝트 문서화

- 요구사항, DB Schema, API 명세 관리
- 회의록 및 트러블슈팅 기록



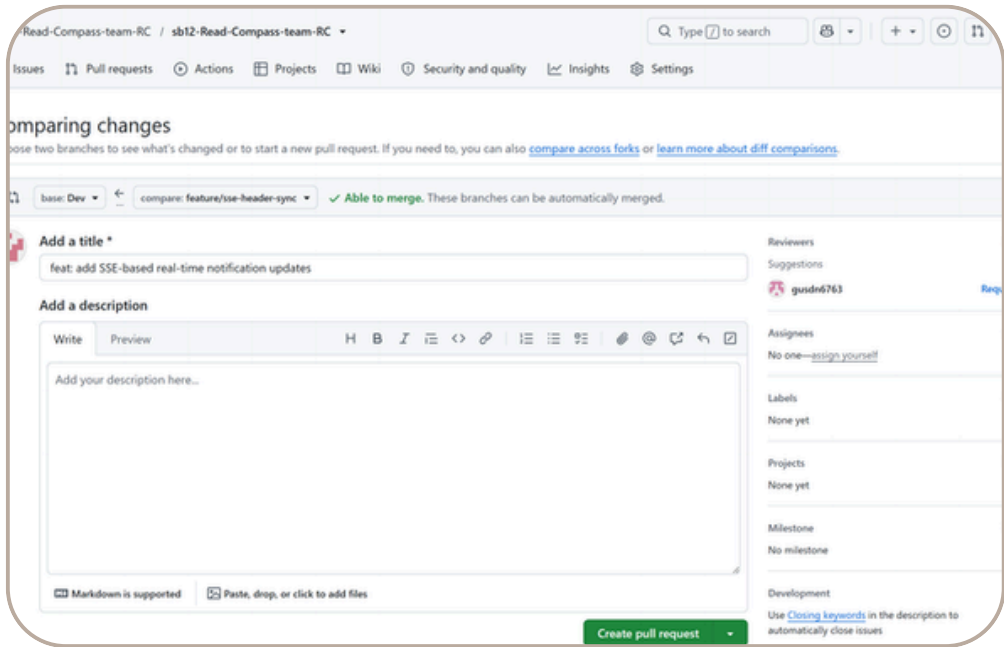
이슈 및 일정 관리

- 스프린트 단위 백로그 생성 및 담당자 지정
- 칸반 보드를 통한 작업 상태 실시간 추적



코드 형상 관리

- Feature Branch 전략
- PR 기반 코드 병합 및 변경 이력 관리



1. ERD

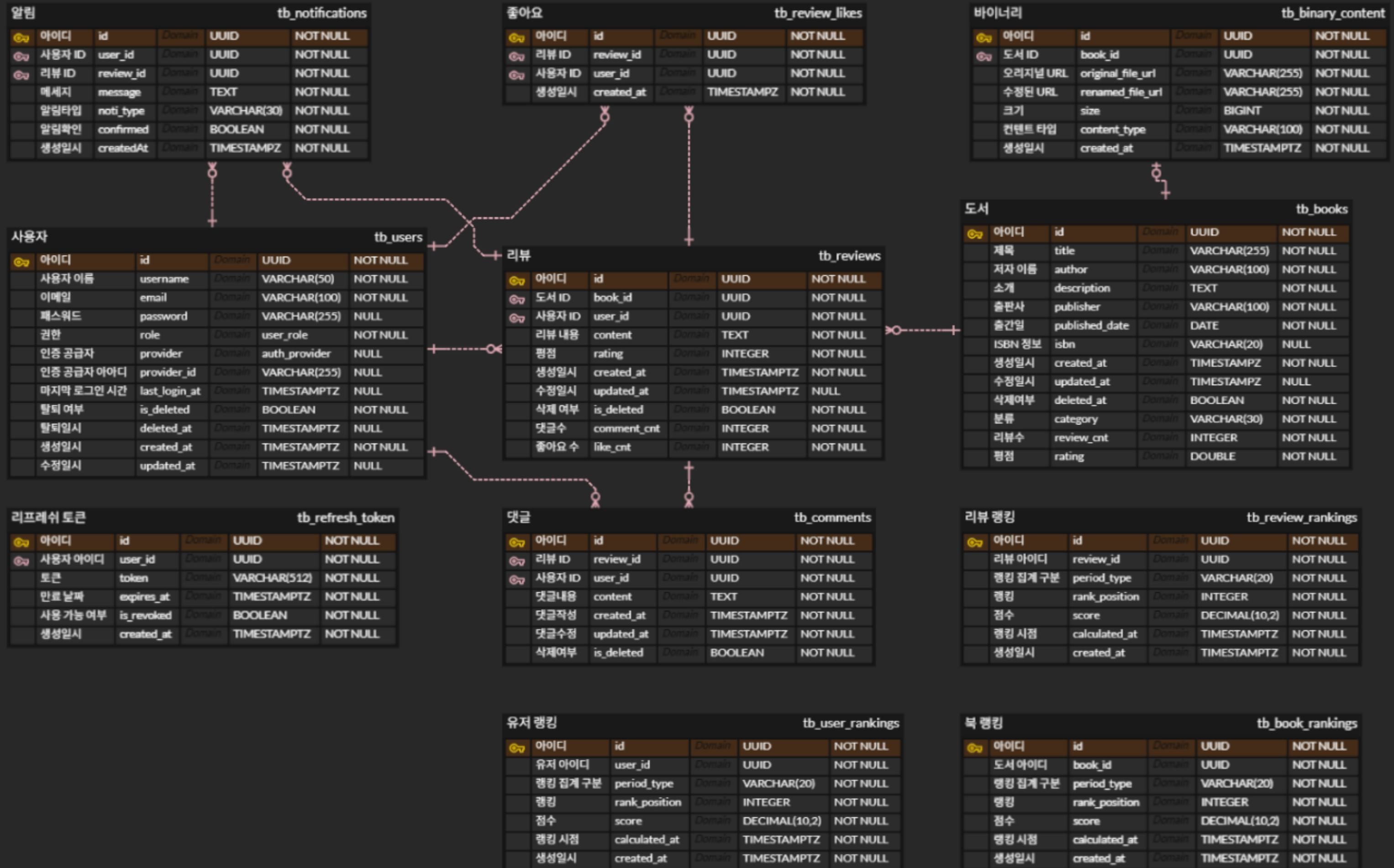
2. 핵심 기능 설계

3. 시스템 구조도

04. ARCHITECTING

ERD

데이터베이스 설계



외부 API 연동을 통한 도서 정보 자동 입력

도서 검색 + OCR 기반 ISBN 매칭

OCR 기반 ISBN 추출 + Naver API 도서 정보 자동 입력

호출 API <https://api.ocr.space/parse/image>

OCR SPACE API

POST /api/books/isbn/ocr

요청값 **image** ISBN이 포함된 도서 이미지

이미지를 OCR Space API로 전송

내부 처리

OCR 결과에서 ISBN 패턴 추출

활용 응답값

추출된 ISBN

ISBN 포함 이미지 업로드

OCR API 호출
이미지 → ISBN

NAVER 책 검색 API 호출
ISBN → 도서정보

등록 폼에 도서 정보 자동 입력

호출 API <https://openapi.naver.com/v1/search/book.json>

NAVER BOOK API

GET /api/books/info?isbn={isbn}

요청값 **isbn** 조회할 도서 ISBN

전달받은 ISBN을
Naver Book API 검색어로 사용

내부 처리

도서 기본 정보와 썸네일 정보를 조회

활용 응답값

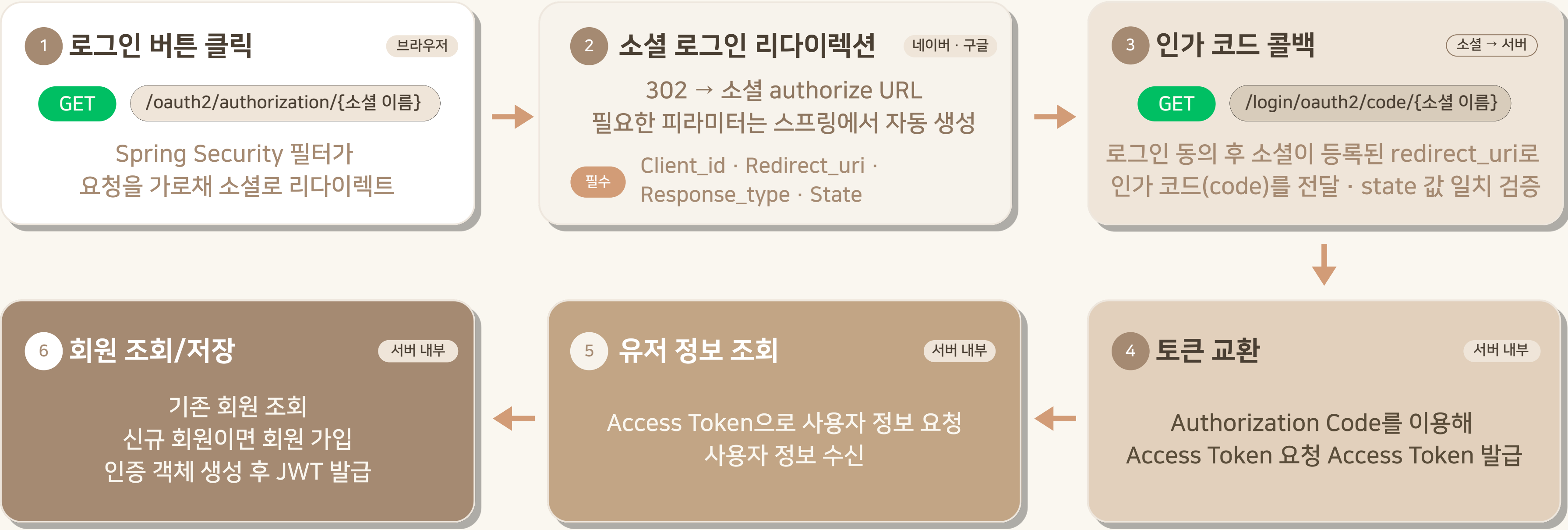
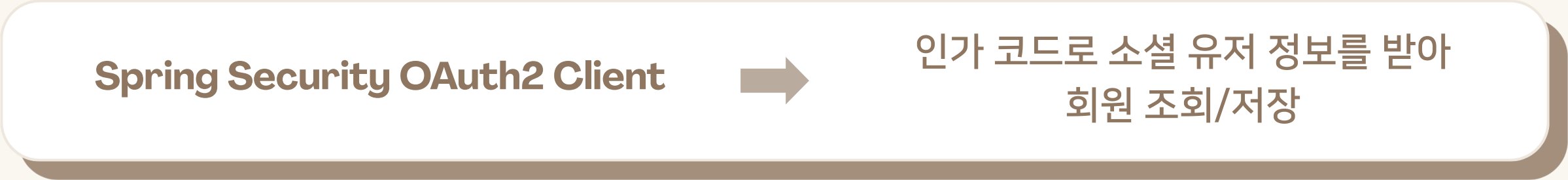
title	author
publisher	publishedDate
description	thumbnailImage

OAuth2 소셜 로그인 API 흐름

로그인 사용 API

네이버 authorize <https://nid.naver.com/oauth2.0/authorize>

구글 authorize <https://accounts.google.com/o/oauth2/v2/auth>



JWT 발급 · 재발급 흐름

토큰 사용

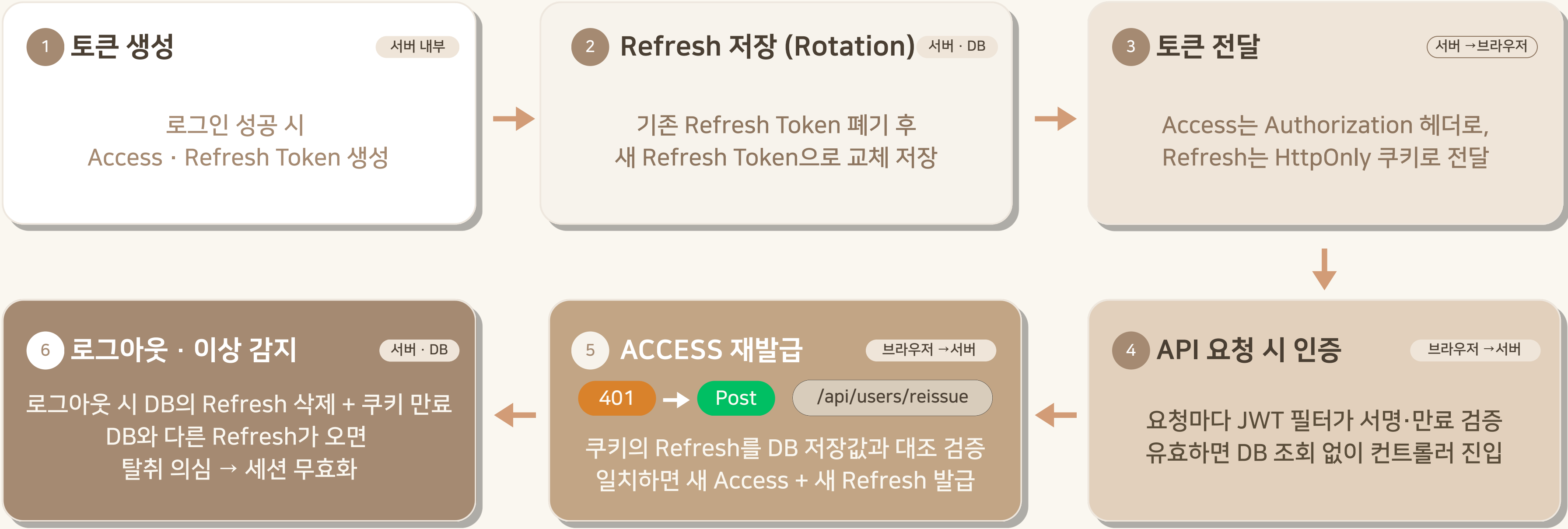
Access Token

프론트 보관 · Authorization: Bearer 헤더로 전송

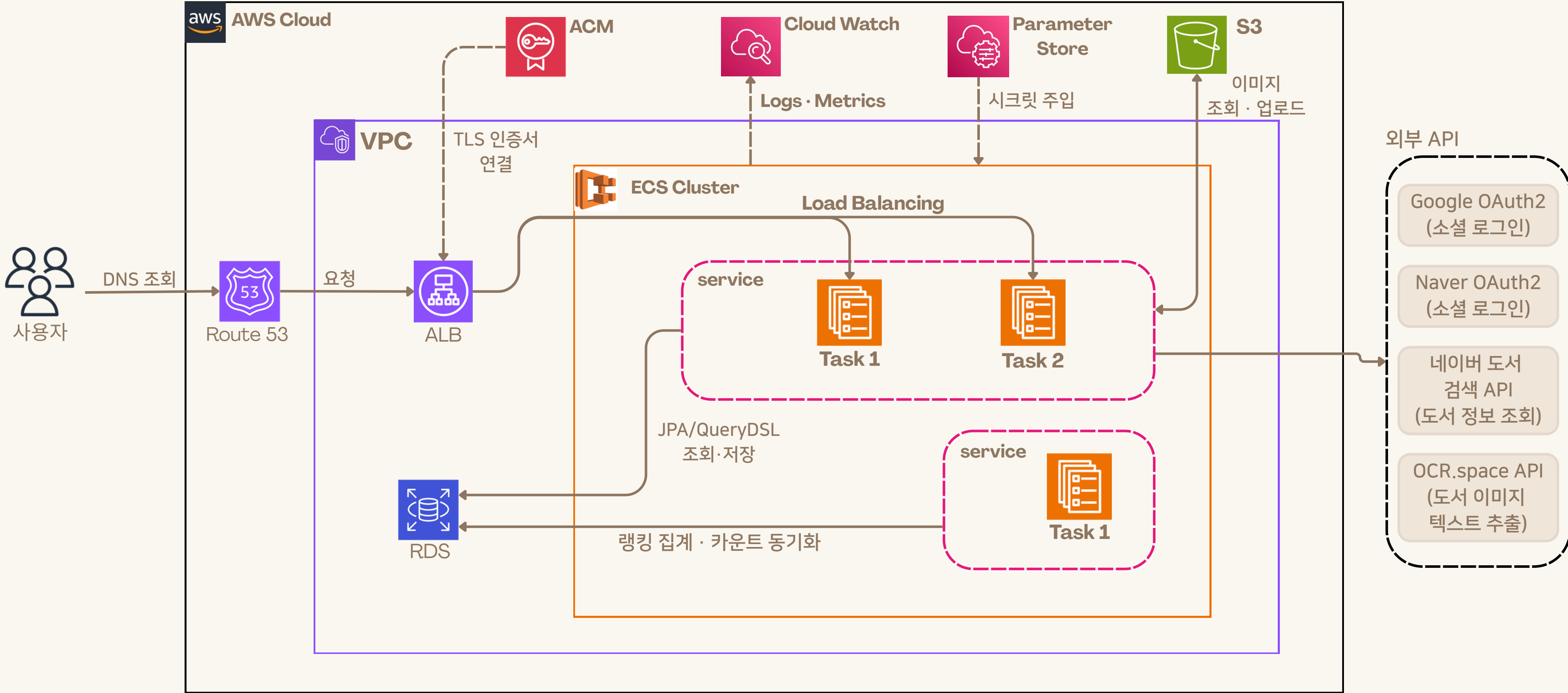
Refresh Token

HttpOnly 쿠키 · JS 접근 불가 · 요청 시 자동 전송

Access + Refresh 이중 토큰 → OAuth2 인증 성공 후 자체 JWT 발급



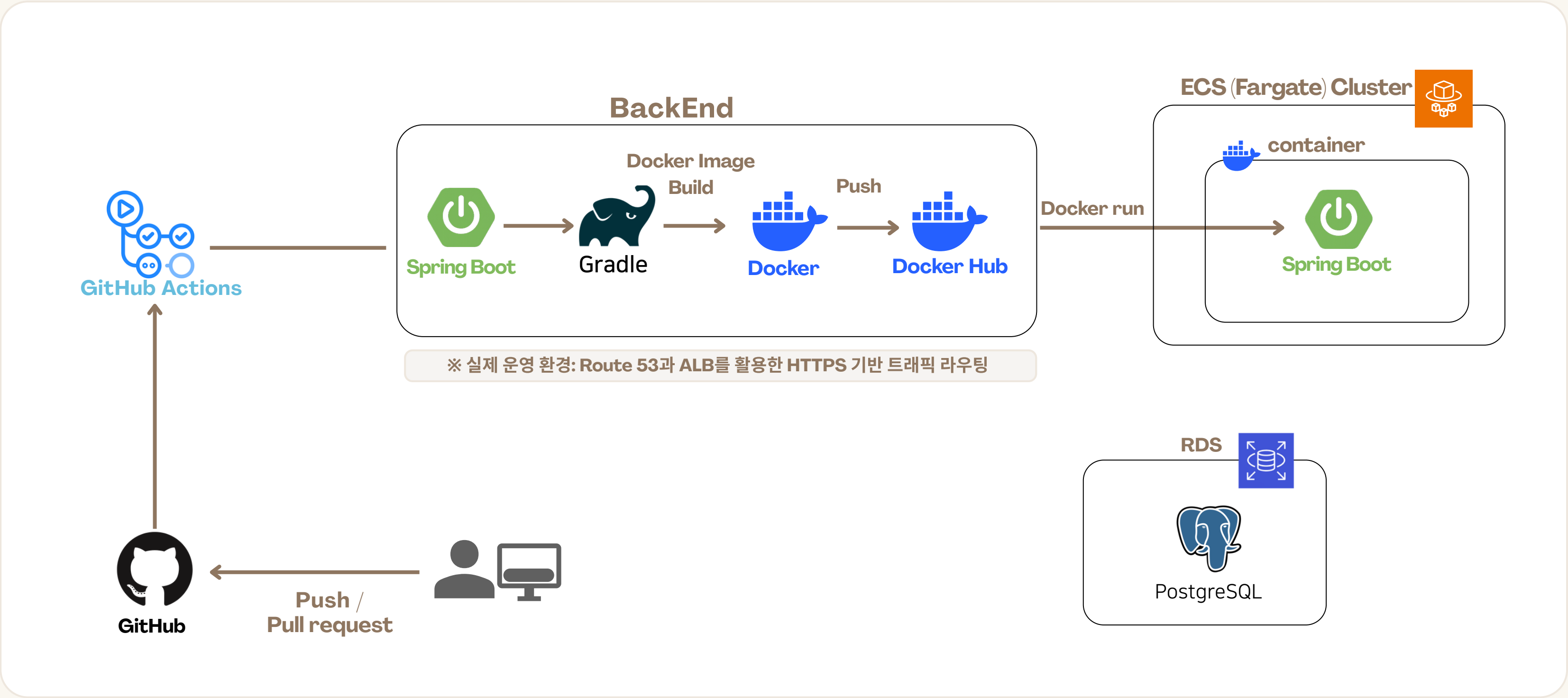
통합 아키텍처



시스템 아키텍처



배포 아키텍처



1.주요 기능 구현

2.시연

3.트러블 슈팅

05. IMPLEMENT

로그인 페이지

01

로그인

데이터베이스에 저장된
사용자 정보를 통해 사용자 인증

02

소셜 로그인

사용자 정보를 통한 사용자 인증을
기존의 소셜 도메인의 인증으로 실행



다시 만나서 반갑습니다!

덕후감에 로그인해주세요

이메일

이메일을 입력해주세요

비밀번호

비밀번호를 입력해주세요

로그인

또는 소셜 계정으로

 Google로 계속하기

 네이버로 계속하기

계정이 없으신가요

회원가입

03

회원가입

회원가입 페이지로 이동

회원가입 페이지

01 회원가입

형식에 맞는 사용자 인증 정보를
사용자 정보 데이터베이스에 등록

만나서 반갑습니다!

가입을 위해 정보를 입력해주세요

이메일

이메일을 입력해주세요

닉네임

닉네임을 입력해주세요

비밀번호

비밀번호를 입력해주세요

비밀번호 확인

한 번 더 입력해주세요

가입하기

또는 소셜 계정으로

Google로 계속하기

N 네이버로 계속하기

계정이 있으신가요? [로그인](#)

만나서 반갑습니다!

가입을 위해 정보를 입력해주세요

이메일

a

유효하지 않은 이메일입니다

닉네임

a

닉네임은 2자 이상 입력해주세요

비밀번호

.

비밀번호는 8~20자이며 영문, 숫자, 특수문자를 포함해야 합니다

비밀번호 확인

...

비밀번호가 일치하지 않습니다

가입하기

또는 소셜 계정으로

Google로 계속하기

N 네이버로 계속하기

계정이 있으신가요? [로그인](#)

02

입력 내용 검증

사용자가 올바른 형식의 정보를 입력을 위한
입력 형식 불일치 안내

Google로 계속하기

N 네이버로 계속하기

계정이 있으신가요? [로그인](#)

이미 존재하는 닉네임입니다.

계정이 있으신가요? [로그인](#)

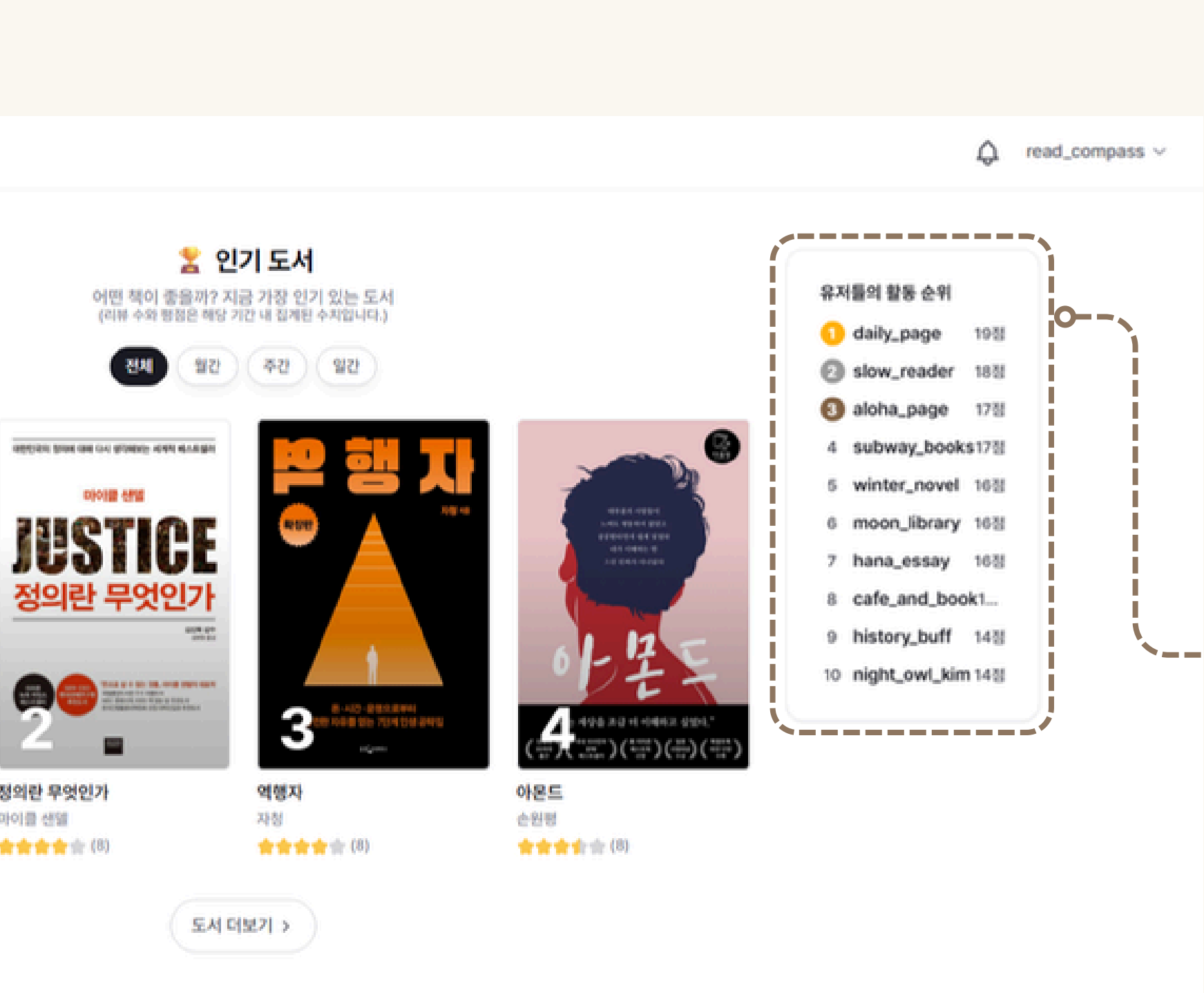
이미 존재하는 이메일입니다.

03

사용자 UNIQUE 보장

UNIQUE 값인 이메일과 닉네임이 중복될 경우
경고하여 이메일과 닉네임 변경을 유도

대시보드 - 사용자 순위



01

파워 유저 랭킹

사용자의 활동 내역에 따라
점수를 계산해 순위 표현



01 - 01

유저 랭킹 모달

화면의 가로축이 좁을 경우
모달 형식으로 표현



인기 도서 점수를 계산하여
순위를 선정한 목록

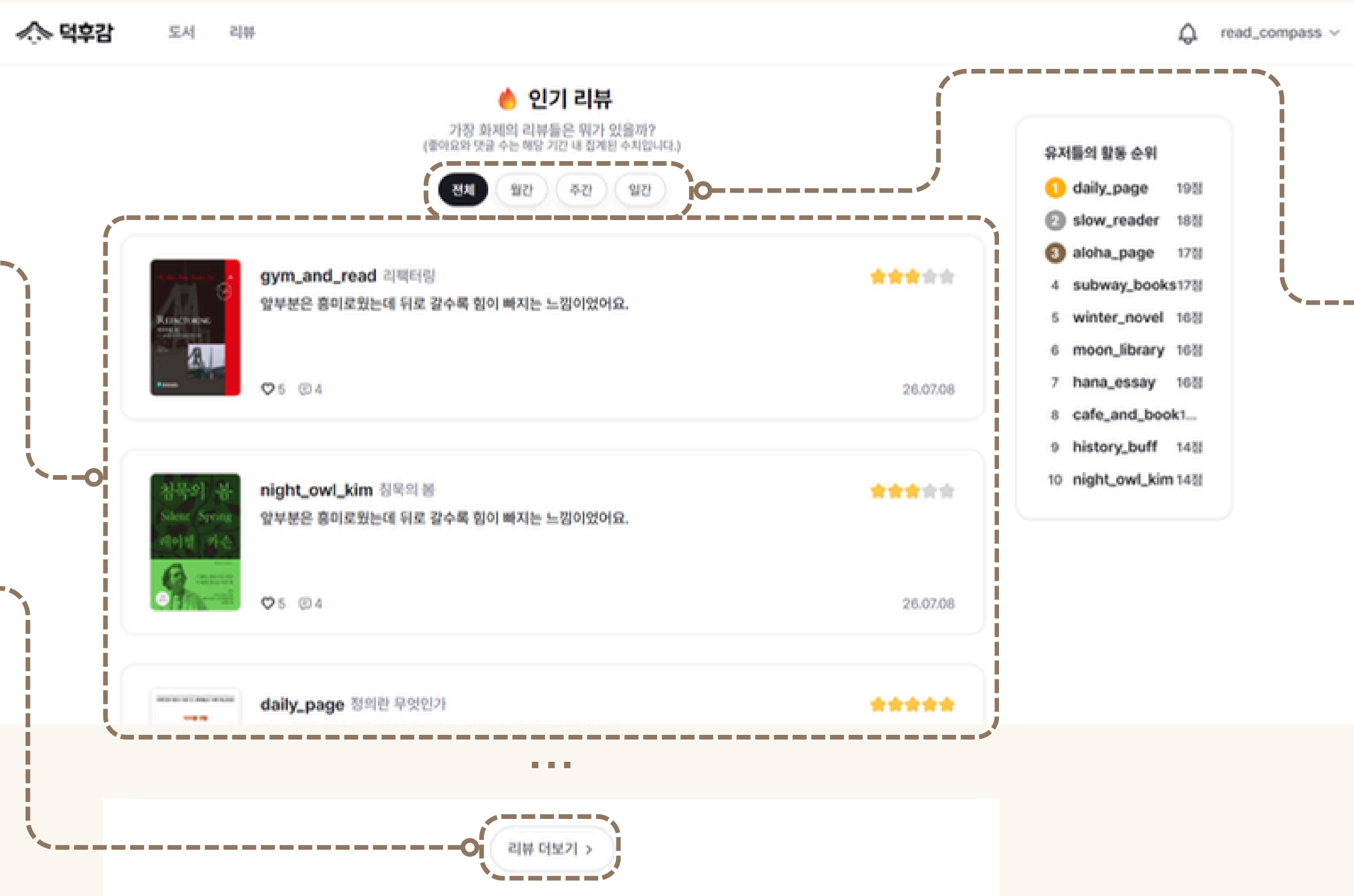
[전체 도서 목록 페이지로 이동](#)

03 기간별 랭킹

도서 랭킹 집계 기간을 선택



대시보드 - 인기 리뷰



도서 페이지

01 키워드 검색 기능

키워드
도서명/저자/
ISBN/카테고리
포함하는 도서 검색

02 전체 도서 목록

도서 썸네일/제목
리뷰 개수
평균 별점
정보를 담은
모든 도서 리스트

03 신규 도서 등록

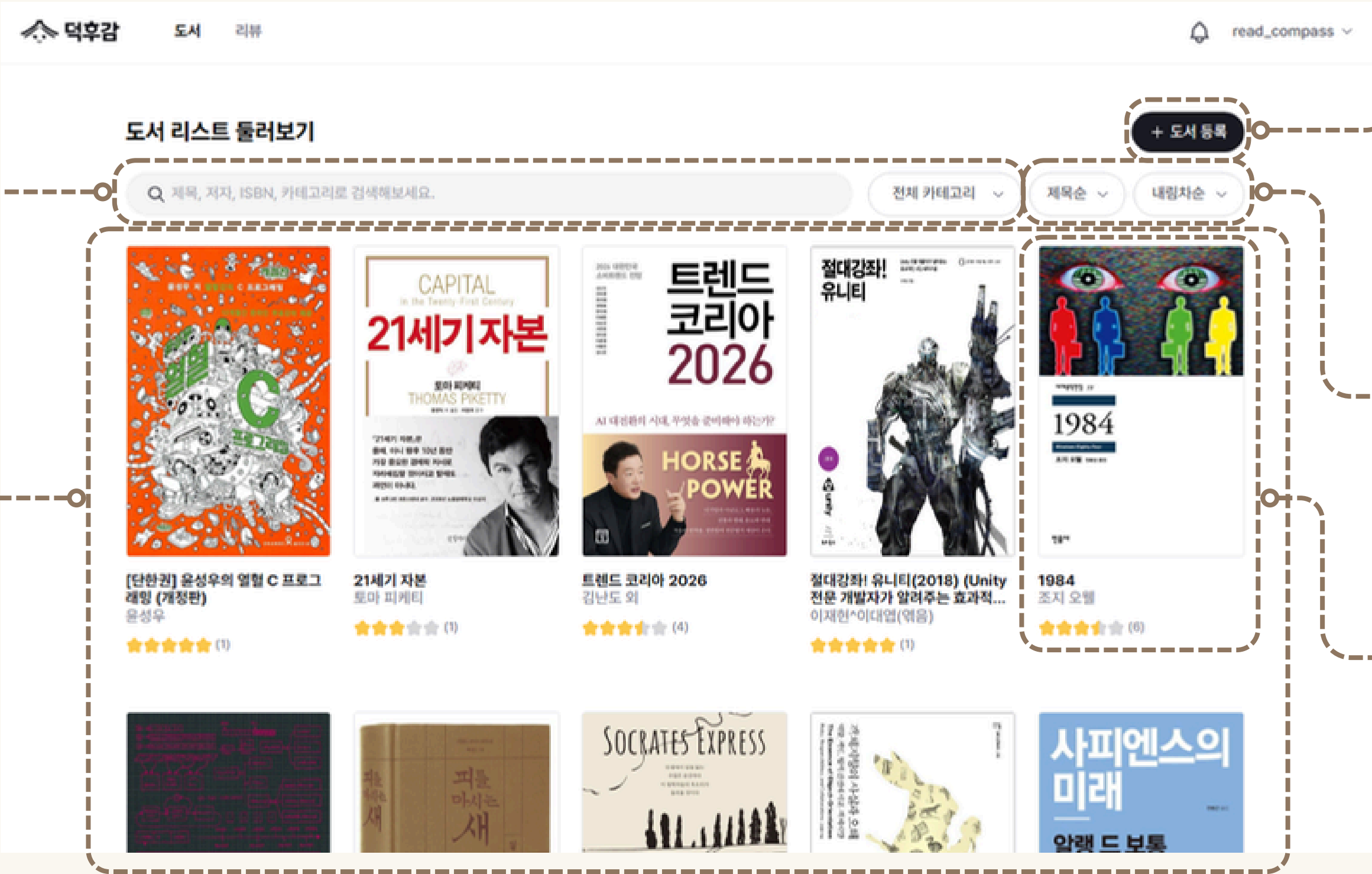
도서 등록 페이지로 이동

04 정렬 기능

제목/출판인/평점/리뷰
오름차순/내림차순
정렬 기준 설정 가능

05 도서 상세보기

해당 도서 상세 페이지 이동



도서 등록 페이지

01 도서 정보 입력

- 도서 카테고리
- ISBN
- 책 표지 사진
- 제목
- 저자
- 출판사
- 출판 년도
- 책 소개글

🏠 덕후감

도서

리뷰

🔔

read_compass

▼

도서 등록하기

카테고리

카테고리 선택

카테고리 선택

소설

시/에세이

인문

가정/육아

🖼️ 이미지로 ISBN 입력하기

정보 불러오기

📄 사진을 업로드해주세요

지은이를 입력해주세요

출판사

출판사를 입력해주세요

출판년도

출판일을 입력해주세요

📅

설명

책에 대한 설명을 입력해주세요

<

2026년 7월

>

일	월	화	수	목	금	토
28	29	30	1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	1

취소

등록

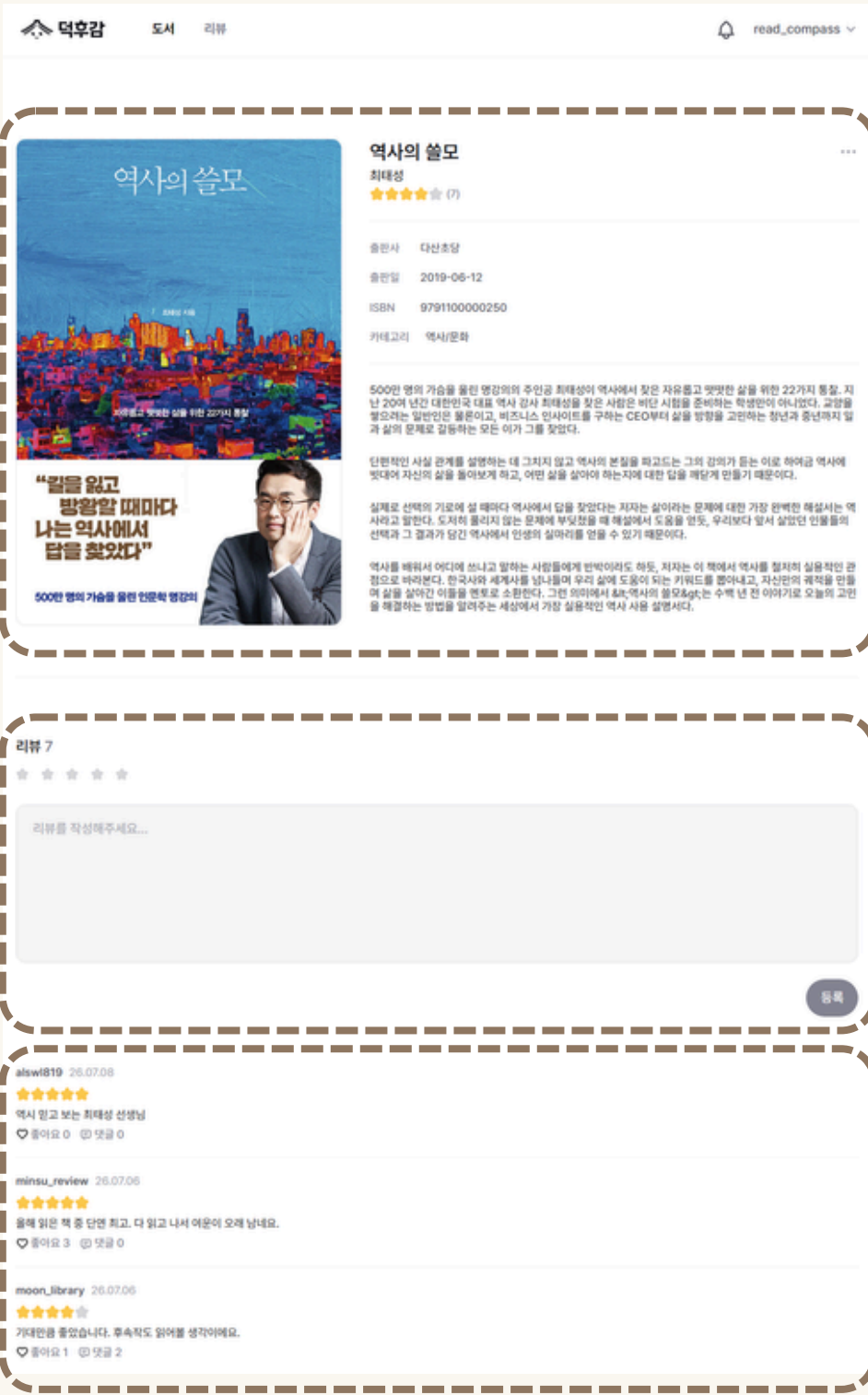
02 이미지 OCR

OCR API를 사용하여
이미지의 ISBN 정보를 추출

03 도서 정보 검색

NAVER OPEN API를 사용하여
ISBN 정보로 도서 정보 검색

도서 상세 페이지



01 도서 상세 정보



02 도서 정보 수정

03 리뷰 목록



04 리뷰 관리

본인이 작성한 리뷰 수정/삭제

05 리뷰 등록 도서에 대한 평점 / 리뷰 작성



리뷰 페이지

01 키워드 검색 필터 기능
도서명/사용자/리뷰 내용으로
검색하여 표시

03 리뷰 목록
도서 썸네일/제목
리뷰
평균 별점
좋아요 수
댓글 수
리뷰 작성 날짜

02 정렬 기능
시간순/평점순,
오름차순/내림차순
정렬 가능

04 리뷰 상세보기
해당 리뷰 상세 페이지 이동

 **덕후감**

도서 리뷰

 alswl819

리뷰 리스트

 도서명, 리뷰어, 내용을 검색해보세요.

시간순

오름차순



winter_novel 정의란 무엇인가

★★★★★

저와는 잘 맞지 않는 책이었습니다. 끝까지 읽기가 힘들었어요.

♡ 5

💬 1

26.01.17



daily_page 시간의 향기

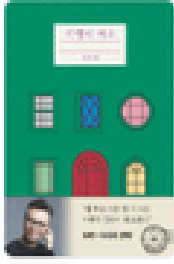
★★★★★

문장이 좋아서 술술 읽힙니다. 결말이 조금 아쉬웠지만 좋았어요.

♡ 1

💬 0

26.01.18



minsu_review 여행의 이유

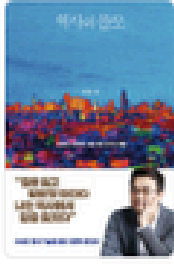
★★★★★

전반적으로 만족스러웠습니다. 중반부가 조금 늘어지긴 하지만 추천해요.

♡ 7

💬 0

26.01.29



daily_page 역사의 쓸모

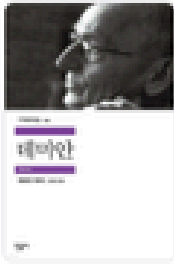
★★★★★

생각할 거리를 많이 던져주는 책입니다. 한 번쯤 읽어볼 만해요.

♡ 2

💬 2

26.01.31



sf_lover_choi 데미안

★★★★★

인생 책이라고 할 만합니다. 읽는 내내 몰입해서 하루 만에 다 읽었어요.

♡ 2

💬 0

26.02.02



history_buff 도메인 주도 설계

★★★★★

기대만큼 좋았습니다. 후속작도 읽어볼 생각이예요.

♡ 1

💬 2

26.02.09

알림 / 사용자 관리



시연

TROUBLE SHOOTING

SSE를 활용한 헤더 알림 실시간 갱신

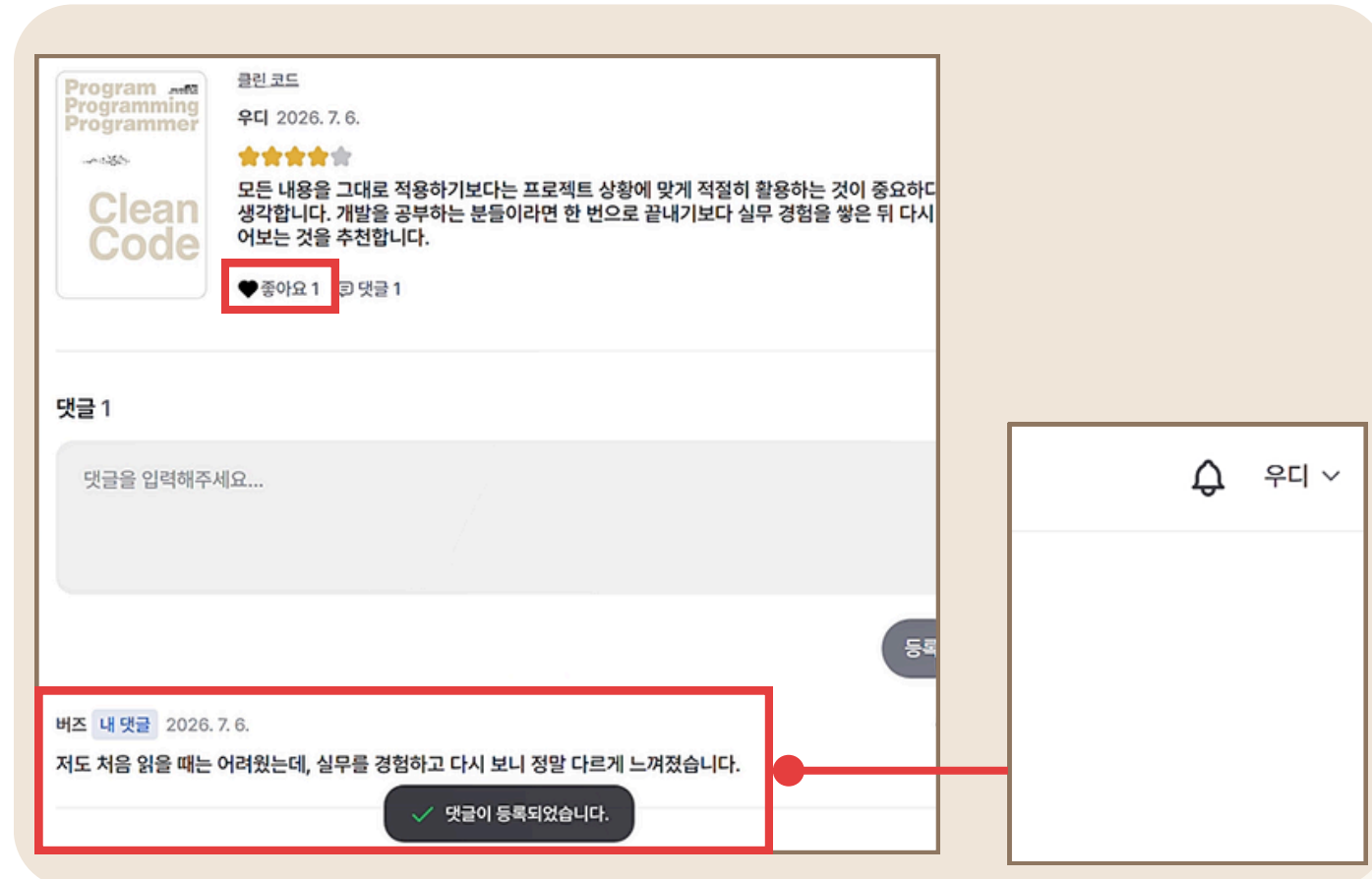
문제 인식

기존 기능: 좋아요, 댓글 등록 시 사용자 알림 제공.

! 발생한 Issue: 새로운 알림이 발생해도

헤더의 알림 아이콘(빨간점)이 즉시 갱신되지 않음.

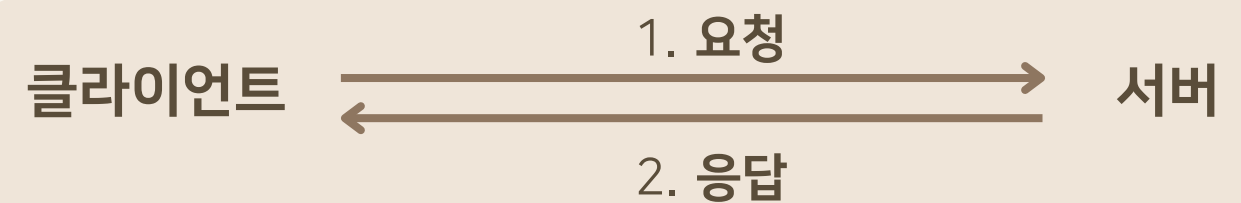
로그인 시, 페이지 새로고침 시, 알림창을 수동으로 열었을 때만
뒤늦게 갱신되는 현상 발생.



원인 분석

“HTTP의 단방향성”

기존 구조는 클라이언트가 요청(REQUEST)을 보내야만
서버가 응답(RESPONSE)을 줄 수 있는 구조



*요청 - 응답 후 연결 종료.

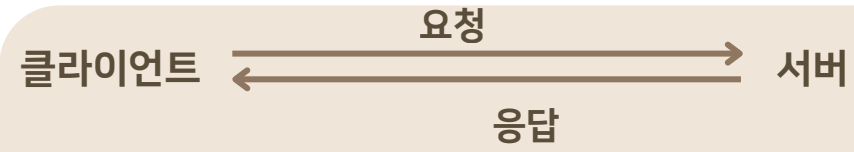
서버에서 새로운 알림(이벤트)이 발생해도 클라이언트가
먼저 묻지 않으면 알림을 보낼 수 없음.

SSE를 활용한 헤더 알림 실시간 갱신

해결 방안

POLLING

: 일정 주기로 서버에 요청



*(EX. 5초마다 반복)

※ 요청 주기 설정 가능

→ 주기가 짧을수록 실시간성 ↑ / 서버 요청량 ↑

WEBSOCKET

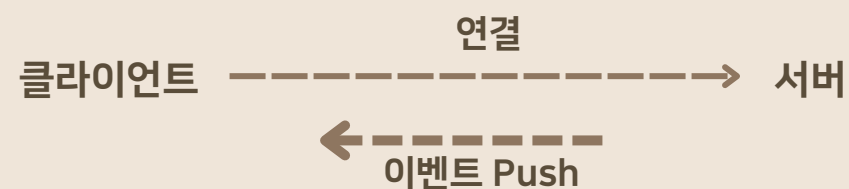
: 양방향 실시간 통신



*양방향 연결 유지

SSE (Server-Sent Events)

: 서버 → 클라이언트 단방향 실시간 전송



*연결 유지

최종 적용>>

① 알림 생성 후 SSE 이벤트 전송

```
notificationSseService.send(receiver.getId());
```

② 브라우저로 notification 이벤트 전송

```
emitter.send(
  SseEmitter.event()
    .name( eventName: "notification")
    .data( object: "new")
);
```

③ 프론트에서 SSE 이벤트 수신

```
onmessage(event : EventSourceMessage ) : void {
  if (event.event === "notification") {
    checkUnreadNotifications();
  }
}
```

④ 최신 알림 목록 조회

```
const response : NotificationsResponse = await getNotifications({
```

⑤ 읽지 않은 알림 여부 확인 후 React 상태(State) 변경

```
const hasUnread : boolean = response.content.some(
  notification : Notification => !notification.confirmed
);
setHasUnreadNotifications(hasUnread);
```

⑥ React 재렌더링으로 빨간 점 즉시 갱신

```
{hasUnreadNotifications && (
  <div className="absolute top-0 right-[-5px] w-1.5 h-1.5 bg-red-500 rounded" />
)}
```

• hasUnreadNotifications가 true이면 빨간 점이 보이고, false이면 사라짐

도서 목록 조회 성능 개선

문제 상황

도서 목록 조회 시 리뷰 수와 평점을 불러오기 위해
Review 테이블을 조인하고 count/avg/groupBy로 매번 집계

리뷰 데이터 증가 시 목록 조회 비용이 커지는 문제

코드 개선

```
NumberExpression<Long> reviewCount = r.id.count();  
NumberExpression<Double> rating = r.rating.avg();
```

```
.from(b)  
.leftJoin(bc).on(bc.book.id.eq(b.id))  
.leftJoin(r).on(  
    r.book.id.eq(b.id)  
    .and(r.deleted.isFalse())  
)
```



```
b.publishedDate,  
b.isbn,  
b.category,  
bc.renamedFileUrl,  
b.reviewCnt,  
b.rating,  
b.createdAt,  
b.updatedAt  
)  
.from(b)  
.leftJoin(bc).on(bc.book.id.eq(b.id))  
.where(where)  
.orderBy(orderBy)
```

- 리뷰 등록/수정/삭제 시점에 Book 엔티티의 reviewCnt와 rating을 갱신하도록 변경
- 도서 목록 조회에서는 Review 테이블을 조인하지 않고 Book의 집계 컬럼을 직접 조회

성능 측정 결과

측정 조건

- 도서 약 1,000건, 리뷰 약 10,000건
- 동일 API 20회 호출
- 초기 5회 워밍업 제외 후 15회 평균

reviewCount 정렬 목록 조회

개선 전 → 개선 후
23.733ms → 13.0147ms
약 45.2% 감소

rating 정렬 목록 조회

- 개선 전: 9.2726ms
 - 개선 후: 8.89493ms
- 약 4.1% 감소

비정규화 카운트 컬럼의 동시성 이슈 대응

문제 인식

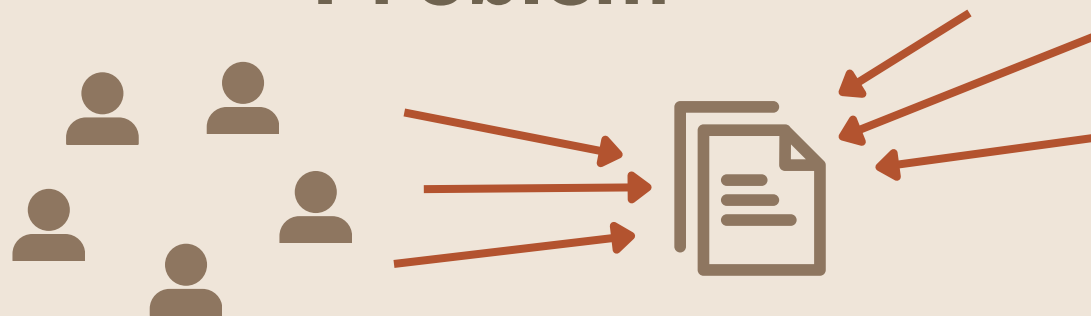
"조회 성능 최적화를 위한 비정규화 데이터의 정합성 의심"

Background



조회 기능의 COUNT 쿼리 비용을 줄이기 위해, 모든 카운트를 테이블 내 비정규화 컬럼으로 관리

Problem



다수의 사용자가 동일한 문서나 리뷰에 동시에 요청
최종 값이 실제 요청 횟수와 일치하지 않고
불안정하게 저장되는 현상 예측

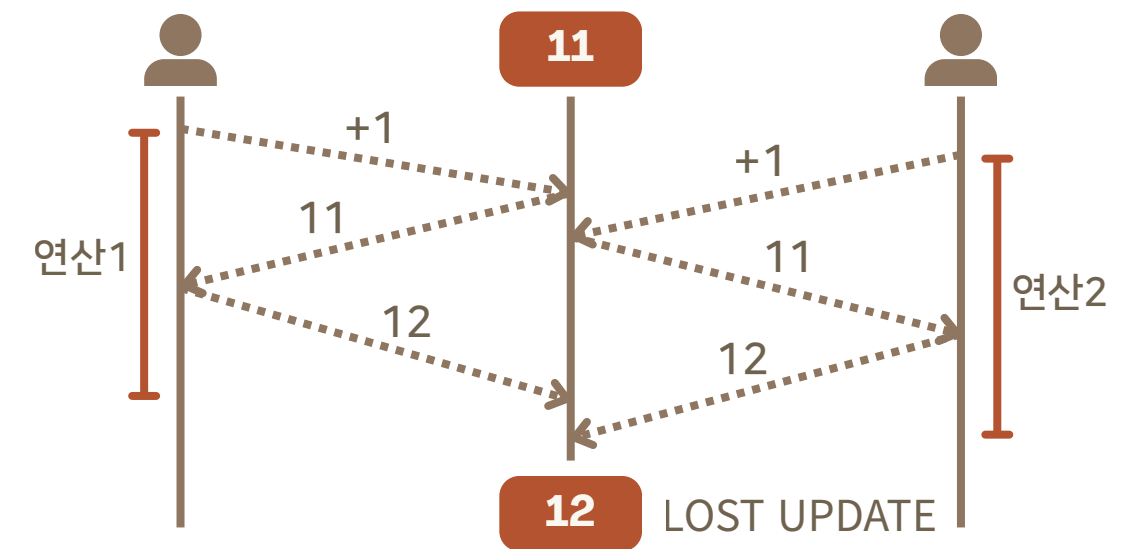
원인 분석

"동일 자원에 대한 동시 접근으로 인한 Race Condition 발생"

Mechanism

여러 트랜잭션이 동일한 카운트 데이터에
동시에 접근(SELECT)하여 연산을 수행.

Execution Failed



레이스 컨디션 발생 가능성 확인

비정규화 카운트 컬럼의 동시성 이슈 대응

해결방안

Optimistic Lock

연산의 전 후 버전을 비교하여 값이 달라질 경우 exception 호출

VS

Pessimistic Lock

연산이 이뤄지는 동안 데이터베이스 접근을 제한하고 연산이 끝난 후 다시 데이터베이스로의 접근을 허용

1. Race Condition의 완전한 차단으로 확실한 정합성 보장
2. Optimistic lock의 불필요한 재시도 비용을 줄이기 위함

최종 적용

- PESSIMISTIC LOCK 적용
- 접근할 count가 존재하는 row에 lock을 적용
- 동일한 row에 대한 수정이 동시에 수정되지 않게 함.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@NotNull
@Query("""
    select r
    from Review r
    where r.id = :reviewId
    and r.deleted = false
    """)
Optional<Review> findActiveByIdForUpdate(UUID reviewId)
```

+ 정합성 보장 배치 프로그램



- 운영 중 예외 상황에 값의 불일치를 대비
- 실제 테이블의 row 수와 count 컬럼 비교 및 보정

JWT 한계점

로그인 유지 방식



JWT 채택

The diagram illustrates the benefits of adopting JWT. It shows that in an ECS environment, JWT can be verified by any server. It also shows that both general login and OAuth2 login can use the same JWT token. Finally, it shows that unlike session-based authentication, JWT does not require a database lookup for each request.

- ECS 환경에서 JWT는 어느 서버든 서명 검증만으로 인증 가능
- 일반 로그인과 OAuth2 로그인을 동일한 JWT 발급 흐름으로 통일
- 세션 방식과 달리 매 요청마다 DB 조회가 필요 없음



문제

토큰 탈취 가능성

헤더로 전송되는 토큰은 노출 시 그대로 인증 수단이 됨

문제1

즉시 무효화 불가

한번 발급된 토큰은 만료 전까지 서버가 강제로 폐기할 수 없음

문제2

JWT 단점 보완

수명

Access 단명

탈취 시 피해 시간을 줄이기 위해
Access Token의 만료 시간을 짧게 설정

```
jwt:  
  access-expire-ms: ${ACCESS_EXPIRE_MS:1800000} # 30분  
  refresh-expire-ms: ${REFRESH_EXPIRE_MS:1209600000} # 14일  
cookie:  
  secure: true  
  same-site: Lax  
  refresh-max-age: ${REFRESH_MAX_AGE:1209600} # 14일(초)
```

구조

이중 토큰 분리

인증은 Access Token
재발급은 Refresh Token - DB 저장
노출 빈도와 수명이 다른 역할을 분리

```
public enum TokenType  
    ACCESS, 2개 사용 위치  
    REFRESH; 6개 사용 위치
```

갱신

Rotation 교체

Access Token 재발급 시
Refresh Token도 새로 발급
이전 Refresh는 DB에서 즉시 무효 처리

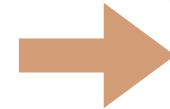
```
public void onAuthenticationSuccess(HttpServletRequest request,  
    HttpServletResponse response,  
    CustomOAuth2User oAuth2User = (CustomOAuth2User) authentication.getPrincipal(),  
    Authentication authentication) {  
    UUID userId = oAuth2User.getUserId();  
    String nickname = oAuth2User.getNickname();  
    String role = oAuth2User.getRole().authority();  
  
    // refresh 발급(Rotation) + HttpOnly 쿠키.  
    tokenIssueService.issueRefresh(response, userId, nickname, role);  
}
```

결과

트레이드오프: JWT의 장점(무상태)은 소폭 양보, 단점(탈취·무효화 불가)은 구조로 최소화
탈취된 ACCESS TOKEN의 유효 시간 최대 30분 · 폐기된 REFRESH는 즉시 거부

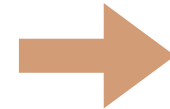
보안 취약점

XSS 공격



악성 스크립트가
브라우저 저장소의 토큰을 읽어
세션 탈취

요청 값 변조



프론트 파라미터를 위조해
타인의 데이터 수정 가능

권한 관리



일반 사용자가
특정 삭제 API를 호출 가능

보안 취약점 대응

토큰 탈취

HttpOnly 저장

Refresh Token을 Cookie에 저장해
JS 접근을 차단

```
public void issueRefresh(HttpServletResponse response, UUID u
    String refresh = jwtUtil.createJwt(TokenType.REFRESH.catego

    Instant expiry = Instant.now().plusMillis(refreshExpireMs);
    refreshTokenService.rotate(userId, refresh, expiry);

    response.addCookie(cookieUtil.createRefresh(refresh));
}
```

값 변조

인증된 헤더 사용

프론트에서 값을 받지 않고
인증된 사용자의 객체 값을 사용

```
// 리뷰 수정
@PatchMapping("/{reviewId}") @ gusdn6763 +1
public ResponseEntity<ReviewDto> updateReview(
    @PathVariable UUID reviewId,
    @AuthenticationPrincipal CustomUserDetails userDe
    @Valid @RequestBody ReviewUpdateRequest request
) {
```

권한 관리

권한 관리

특정 역할을 가진 사용자만
메서드를 호출할 수 있도록 제한

```
// 소프트 딜리트 - deleted_at 업데이트, DB 삭제X (관리자만 가능)
@PreAuthorize("hasRole('ADMIN')") @ minzy108
@DeleteMapping("/{bookId}")
public ResponseEntity<Void> deleteBook(@PathVariable UUID
    bookService.delete(bookId);
    return ResponseEntity.noContent().build();
}
```

DEOK - HUGAM

누군가의 기록이 데이터가 되는 곳

팀명 read-compass

주제 덕후감 프로젝트

깃허브 [github.com/sb12-
Read-Compass-team-RC](https://github.com/sb12-Read-Compass-team-RC)

read_compass ★★★★★

#감사합니다

♡87 💬123

26.07.13